# Computer Graphics

## CSCI 490, CSCI 631

## Project 2 Preparation

### The Graphics Pipeline - Transformations and Simple Objects

---

## Summary

The purpose of this project is to implement some routines in preparation for an assignment to implement the graphics transformation pipeline.

## Things That Ought to be Implemented

This is a list of data structures and routines that need to be implemented. This is not an inclusive list. You may need to implement other routines.

### Data Structures

The data structures need not be C++ classes or C structs, they can be merely conceptual (i.e., " In my code, a transform is always `double [4][4]`" ). You need not use these exact names for your data types, but you should have a way of representing these structures.

- `point` — a 3D point
- `pointh` — a 3D homogeneous point
- `vector` — a 3D vector
- `xform` — a 3D transformation matrix

(Those who are willing to make classes or structs may have a bit extra work up front, but it will definitely pay off as they are used. It's a tradeoff. You choose.

### Functions

The functions listed below are **not** necessarily the function headers you should use. But somehow, you should have the functionality for the following:

#### Points and vectors

- `multiply(point return, float scalar, point p)` — Multiplies a scalar by a point and returns a point.
- `multiply(point return, point p, float scalar)` — Multiplies a point by a scalar and returns a point.

  If you're using C++ operator overloading, you'll need to implement both versions. Otherwise, one will be sufficient.

- `multiply(vector return, float scalar, vector v)` — Multiplies a scalar by a vector and returns a vector.

- `multiply(vector return, vector v, float scalar)` — Multiplies a vector by a scalar and returns a vector.

  Only one of these is necessary unless C++ operator overloading is used.

- `copy(point dest, point src)` — Copies a source point data structure into a destination point data structure.
- `copy(vector dest, vector src)` — Copies a source vector data structure into a destination vector data structure.
- `dot(float return, vector v1, vector v2)` — Computes the dot product of v1 and v2 and returns the scalar value.
- `cross(vector return, vector v1, vector v2)` — Computes the cross product of v1 and v2 and returns the vector value.
- `mag2(float return, vector v)` — Computes the magnitude squared of vector v and returns the scalar value. (The magnitude squared avoids the `sqrt()` and is often useful in its own right. If you want the magnitude, take the square root of the value returned by this function.
- `normalize(vector v)` *or* `normalize(vector return, vector v)` — The first normalizes the vector v in place. The second version returns the normalized version of v, leaving v itself unaltered.
- `add(vector result, vector v1, vector v2)` — Adds two vectors together and returns the vector result.
- `subtract(vector result, vector v1, vector v2)` — Subtracts vector v2 from v1 and returns the vector result.
- `subtract(vector result, point p1, point p2)` — Subtracts point p1 from point p2 to create a vector that is the returned result.
- `convert(pointh return, point p)` — Converts a point into a homogeneous point. (This is a good candidate for something that doesn't necessarily need to be a function on its own. Just know how to do it directly in your code.
- `convert(pointh return, vector v)` — Converts a vector into a homogeneous point.

**Transformations**

- `identity(xform return)` *or* `identity(xform m1)` — The first returns an identity matrix. The second clears the matrix passed in to the identity.
- `multiply(xform return, xform m1, xform m2)` — Multiplies m1 by m2 (m2 on the right) and returns the resulting transformation.
- `multiply(pointh return, xform m1, pointh p1)` — Multiplies m1 by p1 on the right and returns the resulting homogeneous point.
- `push(xform m1)` — Pushes the given transform onto a stack of transforms.
- `pop(return xform)` — Returns a transformation from the top of the transformation stack.

# Pass off

This sub project will not be passed off. It will not be graded. It's just a laundry list of things that you need to have in order to get the second project working. Come up with your own test cases. Make sure that everything works to your satisfaction.